

ML FOR MECH ENG (CoEP)

Yogesh Haribhau Kulkarni

Outline

1 SESSION 20: ME APPS

About Me

Yogesh Haribhau Kulkarni

Bio:

- ▶ 20+ years in CAD/Engineering software development
- ▶ Got Bachelors, Masters and Doctoral degrees in Mechanical Engineering (specialization: Geometric Modeling Algorithms).
- ▶ Currently doing Coaching in fields such as Data Science, Artificial Intelligence Machine-Deep Learning (ML/DL) and Natural Language Processing (NLP).
- ▶ Feel free to follow me at:
 - ▶ Github (github.com/yogeshhk)
 - ▶ LinkedIn (www.linkedin.com/in/yogeshkulkarni/)
 - ▶ Medium (yogeshharibhaukulkarni.medium.com)
 - ▶ Send email to yogeshkulkarni at yahoo dot com



Office Hours:
Saturdays, 2 to 5pm
(IST); Free-Open to all;
email for appointment.

Applications to Mechanical Engineering

The Announcement

“Revitalizing manufacturing through AI”

- ▶ AI is already transforming the IT industry.
- ▶ It is now time to for an AI-powered society.
- ▶ We will initially focus on the manufacturing industry.

(Reference: “Medium” - Andrew Ng, Dec 14,2017)

Why Manufacturing?

- ▶ Manufacturing touches every part of society.
- ▶ Need to help not just bits and pixels but physical world.

Challenges facing manufacturing

- ▶ Variable quality and yield,
- ▶ Inflexible production line design
- ▶ Inability to manage capacity
- ▶ and rising production costs.

ML to help address them.

Challenges facing manufacturing

More help in

- ▶ Shorten design cycles,
- ▶ Remove supply-chain bottlenecks,
- ▶ Reduce materials and energy waste,
- ▶ and improve production yields.

Why now?

“Industry today is experiencing a never seen increase in available data”

Sources:

- ▶ Environmental data
- ▶ Sensor data from production line
- ▶ Machine tool parameters
- ▶ Logistics, transaction data
- ▶ Customer demand data.

(Reference: What is smart manufacturing? Time Magazine - Chand & Davis, 2010)

How ML can help?

- ▶ Ability to handle high-dimensional problems
- ▶ SVM handling high dimensionality (> 1000) very well.
- ▶ Concerns: Over-fitting

(Ref: "Machine learning in manufacturing: advantages, challenges, and applications"- Thorsten Wuest, et al)

How ML can help?

- ▶ Ability to reduce complexity
- ▶ Present transparent and concrete advice for practitioners
- ▶ E.g. monitor XX and parameter YY at checkpoint ZZ
- ▶ ML finds important features, thresholds

(Ref: "Machine learning in manufacturing: advantages, challenges, and applications"- Thorsten Wuest, et al)

How ML can help?

- ▶ Ability to adapt to changing environment
- ▶ New data, new training, new model.

(Ref: "Machine learning in manufacturing: advantages, challenges, and applications"- Thorsten Wuest, et al)

New wave: IoT

“Internet of Things”

- ▶ IOT is connecting things to the Internet
- ▶ Not the same as connecting things to the cellular network!
- ▶ IOT personal (Health apps)
- ▶ IOT Machine (Machine 2 Machine M2M)
- ▶ IoT Everywhere (5G, wifi)

New wave: IoT

IoT Data

- ▶ Sensors everywhere.
- ▶ By 2020, 50 billion connected devices.
- ▶ Wearables, cars, machines, cities.

New wave: IoT

IoT and Machine Learning

- ▶ Basic idea of machine learning is to build a mathematical model based on training data(learning stage) , predict results for new data(prediction stage) and tweak the model based on new conditions
- ▶ IoT creates a lot of contextual data
- ▶ Use the scale of IoT data to gain new insights

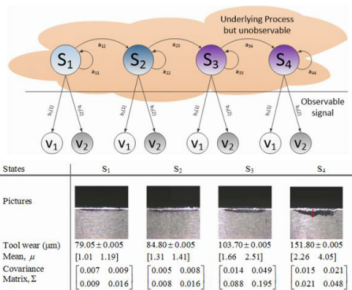
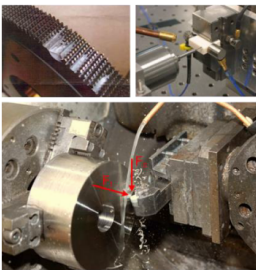
New wave: IoT

IoT ML Applications

- ▶ Consumer : bio sensors(real time tracking)
- ▶ Enterprise : Complex machinery (preventative maintenance), asset efficiency.
- ▶ Public infrastructure: Smart Cities, Dynamically adjust traffic lights

Sample Applications: Tool Wear Estimation

Hidden Markov Model



S. Lee, L. Li*, and J. Ni, 2010, "Online Degradation Assessment and Adaptive Fault Detection Using Modified Hidden Markov Model," ASME Journal of Manufacturing Science and Engineering, 132(2), pp. 021010-11. [PDF]

Ideas for new applications

Project ideas?

- ▶ Kaggle: <https://www.kaggle.com/datasets>
- ▶ UCI: <http://archive.ics.uci.edu/ml/index.php>
- ▶ Other: <http://deeplearning.net/datasets/>

Linear Regression — House Price Prediction

(Ref: scikit-learn User Guide — Generalized Linear Models)

Problem Info

- ▶ Dataset: California Housing (`sklearn.datasets.fetch_california_housing`, built-in)
- ▶ Features: median income, house age, avg rooms, avg bedrooms, population, latitude, longitude
- ▶ Target: Median house value (in 100,000s USD)
- ▶ Goal: Predict house prices using Linear Regression; compare Ridge and Lasso regularization

Import Libraries

```
1 import numpy as np
import pandas as pd
3 import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
5 from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
7 from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Load and Explore Data

```
data = fetch_california_housing()
2 df = pd.DataFrame(data.data, columns=data.feature_names)
df['MedHouseVal'] = data.target
4
print(df.shape)          # (20640, 9)
6 print(df.describe())
print(df.head())
```

Train Linear Regression

```
1 X_train, X_test, y_train, y_test = train_test_split(  
    data.data, data.target, test_size=0.2, random_state=42)  
3  
    scaler = StandardScaler()  
5 X_train = scaler.fit_transform(X_train)  
    X_test = scaler.transform(X_test)  
7  
    lr = LinearRegression()  
9 lr.fit(X_train, y_train)  
    y_pred = lr.predict(X_test)  
11  
    print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))  
13 print("R2  :", r2_score(y_test, y_pred))
```


Compare Regularization

```
1 for name, model in [("Linear", LinearRegression()),  
2                     ("Ridge", Ridge(alpha=1.0)),  
3                     ("Lasso", Lasso(alpha=0.1))]:  
4     model.fit(X_train, y_train)  
5     pred = model.predict(X_test)  
6     rmse = np.sqrt(mean_squared_error(y_test, pred))  
7     r2 = r2_score(y_test, pred)  
8     print(f"{name:8s} RMSE={rmse:.4f} R2={r2:.4f}")
```

Predicted vs Actual

Points close to the diagonal indicate accurate predictions.

```
plt.figure(figsize=(6, 5))
2 plt.scatter(y_test, y_pred, alpha=0.3, s=10)
plt.plot([y_test.min(), y_test.max()],
4         [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Price")
6 plt.ylabel("Predicted Price")
plt.title("Linear Regression: Predicted vs Actual")
8 plt.tight_layout()
plt.show()
```

SVM — Handwritten Digit Recognition

(Ref: scikit-learn SVM example — Digit Classification)

Problem Info

- ▶ Dataset: Digits (`sklearn.datasets.load_digits`, built-in)
- ▶ 1,797 images of handwritten digits (0–9), each 8×8 pixels = 64 features
- ▶ Goal: Classify each image into one of 10 digit classes using SVM
- ▶ Demonstrates the power of SVM's RBF kernel for non-linear image classification

Import Libraries

```
1 import numpy as np
import matplotlib.pyplot as plt
3 from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
5 from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

Load and Visualize Data

```
digits = load_digits()
2 print("Data shape:", digits.data.shape)    # (1797, 64)
  print("Classes   :", digits.target_names)  # [0 1 2 ... 9]
4
6 fig, axes = plt.subplots(2, 5, figsize=(10, 4))
  for ax, img, label in zip(axes.ravel(),
                             digits.images, digits.target):
8     ax.imshow(img, cmap='gray_r')
     ax.set_title(str(label))
10    ax.axis('off')
  plt.suptitle("Sample Digits")
12 plt.show()
```

Train SVM Classifier

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     digits.data, digits.target,
3     test_size=0.2, random_state=42)
4
5 scaler = StandardScaler()
6 X_train = scaler.fit_transform(X_train)
7 X_test  = scaler.transform(X_test)
8
9 svm = SVC(kernel='rbf', C=10, gamma=0.001)
10 svm.fit(X_train, y_train)
11 y_pred = svm.predict(X_test)
12
13 print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
1 cm = confusion_matrix(y_test, y_pred)
  plt.figure(figsize=(8, 6))
3  sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
               xticklabels=digits.target_names,
5               yticklabels=digits.target_names)
  plt.xlabel("Predicted")
7  plt.ylabel("True")
  plt.title("SVM Digit Recognition: Confusion Matrix")
9  plt.tight_layout()
  plt.show()
```


Visualize Misclassifications

```
1 wrong = np.where(y_pred != y_test)[0]
2 fig, axes = plt.subplots(2, 5, figsize=(10, 4))
3 for ax, idx in zip(axes.ravel(), wrong[:10]):
4     ax.imshow(X_test[idx].reshape(8, 8), cmap='gray_r')
5     ax.set_title(f"True:{y_test[idx]} Pred:{y_pred[idx]}")
6     ax.axis('off')
7 plt.suptitle("Misclassified Digits")
8 plt.tight_layout()
9 plt.show()
```

K-Means Clustering — Customer Segmentation

(Ref: scikit-learn Clustering User Guide)

Problem Info

- ▶ Scenario: A retail company wants to segment its customers for targeted marketing
- ▶ Features: Annual Income (k USD) and Spending Score (1–100)
- ▶ Data: Synthetic dataset mimicking a retail loyalty programme
- ▶ Goal: Discover natural customer groups using K-Means; choose optimal K with the Elbow method

Import Libraries and Generate Data

```
1 import numpy as np
  import pandas as pd
3 import matplotlib.pyplot as plt
  from sklearn.cluster import KMeans
5 from sklearn.preprocessing import StandardScaler

7 np.random.seed(42)
  # Simulate 3 customer segments: budget, standard, premium
9 centres = [(30, 20), (55, 55), (80, 85)]
  X = np.vstack([
11     np.random.randn(120, 2) * [8, 10] + c
      for c in centres])
13 df = pd.DataFrame(X, columns=['AnnualIncome', 'SpendingScore'])
  print(df.describe())
```

Elbow Method: Choosing K

```
1 scaler = StandardScaler()
2 X_scaled = scaler.fit_transform(df)
3
4 inertia = []
5 K_range = range(1, 10)
6 for k in K_range:
7     km = KMeans(n_clusters=k, random_state=42, n_init=10)
8     km.fit(X_scaled)
9     inertia.append(km.inertia_)
10
11 plt.plot(K_range, inertia, marker='o')
12 plt.xlabel("Number of Clusters K")
13 plt.ylabel("Inertia (WCSS)")
14 plt.title("Elbow Method")
15 plt.grid(True); plt.show()
```

Fit K-Means with Optimal K

```
1 km = KMeans(n_clusters=3, random_state=42, n_init=10)
  df['Segment'] = km.fit_predict(X_scaled)
3
  labels = {0: 'Budget', 1: 'Standard', 2: 'Premium'}
5 colours = ['#e07b54', '#5b9bd5', '#70ad47']

7 plt.figure(figsize=(7, 5))
  for seg, (name, colour) in zip([0, 1, 2],
9                               zip(labels.values(), colours)):
      mask = df['Segment'] == seg
11     plt.scatter(df.loc[mask, 'AnnualIncome'],
                  df.loc[mask, 'SpendingScore'],
13                 label=name, color=colour, alpha=0.7)
  plt.xlabel("Annual Income (k USD)")
15 plt.ylabel("Spending Score")
  plt.title("Customer Segments (K-Means, K=3)")
17 plt.legend(); plt.tight_layout(); plt.show()
```

Segment Profiles

- ▶ **Budget:** low income, low spending — price-sensitive customers
- ▶ **Standard:** mid income, mid spending — core customer base
- ▶ **Premium:** high income, high spending — loyalty programme targets

```
1 profile = df.groupby('Segment')[  
    ['AnnualIncome', 'SpendingScore']].mean()  
3 profile.index = [labels[i] for i in profile.index]  
print(profile.round(1))
```

PCA + K-Means — Handwritten Digit Clustering

(Ref: scikit-learn Decomposition and Clustering User Guide)

Problem Info

- ▶ Dataset: Digits (`sklearn.datasets.load_digits`, built-in) — 1,797 images, 64 features each
- ▶ Part A: Use PCA to reduce 64 dimensions to 2; visualise digit clusters in 2D
- ▶ Part B: Apply K-Means ($K=10$) on PCA-reduced data; measure cluster quality
- ▶ Part C: Vary the number of PCA components (2, 10, 30) and observe effect on clustering

Import Libraries

```
import numpy as np
2 import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
4 from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
6 from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score
```

Part A: PCA Visualisation

```
1 digits = load_digits()
  X = StandardScaler().fit_transform(digits.data)
3
  pca2 = PCA(n_components=2)
5  X2 = pca2.fit_transform(X)
7
  plt.figure(figsize=(8, 6))
  scatter = plt.scatter(X2[:, 0], X2[:, 1],
8                      c=digits.target, cmap='tab10',
9                      s=10, alpha=0.7)
11 plt.colorbar(scatter, label='Digit')
  plt.title("Digits projected to 2 PCA components")
13 plt.xlabel("PC 1"); plt.ylabel("PC 2")
  plt.tight_layout(); plt.show()
15
  print("Variance explained:", pca2.explained_variance_ratio_.sum())
```

Part B: K-Means on PCA Data

- ▶ **Silhouette score:** measures cluster compactness (higher is better, max 1.0)
- ▶ **Adjusted Rand Index:** compares clusters to true digit labels (1.0 = perfect)

```
for n_comp in [2, 10, 30]:  
2   Xr = PCA(n_components=n_comp).fit_transform(X)  
   km = KMeans(n_clusters=10, random_state=42, n_init=10)  
4   labels = km.fit_predict(Xr)  
   sil = silhouette_score(Xr, labels)  
6   ari = adjusted_rand_score(digits.target, labels)  
   print(f"PCA={n_comp:2d}  Silhouette={sil:.3f}  "  
8         f"ARI={ari:.3f}")
```

Assignment Tasks

- ▶ Report the variance explained by 2, 10, and 30 PCA components
- ▶ Plot the explained variance ratio (scree plot) and identify the “elbow”
- ▶ At which number of PCA components does ARI peak? Explain why
- ▶ Visualise cluster centres (inverse-transform to pixel space) for the best configuration
- ▶ What do the cluster centres look like? Do they resemble recognisable digits?

Assignments for Mechanical Engineering

Predictive Maintenance using Regression Analysis

- ▶ **Dataset:** NASA Turbofan Engine Degradation Simulation Data Set
- ▶ **Source:** NASA Prognostics Data Repository
- ▶ **Description:** Predict the remaining useful life of turbofan engines using regression analysis based on sensor data.

```
import pandas as pd
2 from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
4
data = pd.read_csv('turbofan_data.csv')
6 X = data[['feature1', 'feature2', 'feature3']]
y = data['remaining_useful_life']
8
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
10 model = LinearRegression()
model.fit(X_train, y_train)
12 predictions = model.predict(X_test)
```

Structural Health Monitoring using Anomaly Detection

- ▶ **Dataset:** Bridge Health Monitoring Dataset
- ▶ **Source:** UCI Machine Learning Repository
- ▶ **Description:** Detect anomalies indicating potential structural failures in bridges based on sensor readings.

```
import pandas as pd
2 from sklearn.ensemble import IsolationForest

4 data = pd.read_csv('bridge_health_data.csv')
model = IsolationForest(contamination=0.1)
6 model.fit(data)
anomalies = model.predict(data)
8
```


Design Optimization using Genetic Algorithms

- ▶ **Dataset:** Engineering Design Optimization Dataset
- ▶ **Source:** Kaggle
- ▶ **Description:** Optimize engineering designs using genetic algorithms to evaluate performance metrics.

```
1 from deap import base, creator, tools, algorithms
3 creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
  creator.create("Individual", list, fitness=creator.FitnessMin)
5
  toolbox = base.Toolbox()
7 # Define operations for crossover, mutation, etc.
  population = toolbox.population(n=50)
9 algorithms.eaSimple(population, toolbox, cxpb=0.5, mutpb=0.2, ngen=40)
```

Fault Detection in Rotating Machinery using SVM

- ▶ **Dataset:** Machinery Fault Diagnosis Dataset
- ▶ **Source:** UCI Machine Learning Repository
- ▶ **Description:** Classify faults in rotating machinery using support vector machines (SVM).

```
1 import pandas as pd
2 from sklearn.svm import SVC
3 from sklearn.model_selection import train_test_split
4
5 data = pd.read_csv('machinery_fault_data.csv')
6 X = data.drop('fault_type', axis=1)
7 y = data['fault_type']
8
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
10 model = SVC()
11 model.fit(X_train, y_train)
12 accuracy = model.score(X_test, y_test)
13
```

Thermal Analysis using Neural Networks

- ▶ **Dataset:** Thermal Conductivity Dataset
- ▶ **Source:** Kaggle
- ▶ **Description:** Predict thermal conductivity of materials using neural networks.

```
1 from keras.models import Sequential
2 from keras.layers import Dense
3
4 model = Sequential()
5 model.add(Dense(64, activation='relu', input_dim=10))
6 model.add(Dense(32, activation='relu'))
7 model.add(Dense(1))
8
9 model.compile(optimizer='adam', loss='mean_squared_error')
10
```

Quality Control in Manufacturing using CNNs

- ▶ **Dataset:** Manufacturing Defect Dataset
- ▶ **Source:** Kaggle
- ▶ **Description:** Train a CNN to classify manufacturing defects based on part images.

```
1 from keras.models import Sequential
2 from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
3
4 model = Sequential()
5 model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
6 model.add(MaxPooling2D(pool_size=(2,2)))
7 model.add(Flatten())
8 model.add(Dense(128, activation='relu'))
9 model.add(Dense(10, activation='softmax'))
10
11 model.compile(optimizer='adam', loss='categorical_crossentropy',
               metrics=['accuracy'])
```

Energy Consumption Prediction using Time Series Analysis

- ▶ **Dataset:** Energy Consumption Dataset
- ▶ **Source:** UCI Machine Learning Repository
- ▶ **Description:** Forecast energy consumption patterns using ARIMA models.

```
1 import pandas as pd
  from statsmodels.tsa.arima.model import ARIMA
3
4 data = pd.read_csv('energy_consumption.csv')
5 model = ARIMA(data['consumption'], order=(5,1,0))
  model_fit = model.fit()
7 forecast = model_fit.forecast(steps=10)
```

Material Property Prediction using Gaussian Processes

- ▶ **Dataset:** Materials Project Database
- ▶ **Source:** Materials Project
- ▶ **Description:** Predict material properties using Gaussian process regression.

```
1 from sklearn.gaussian_process import GaussianProcessRegressor
3 gp = GaussianProcessRegressor()
  gp.fit(X_train, y_train)
5 predictions = gp.predict(X_test)
```

Load Prediction in Structural Engineering using Random Forests

- ▶ **Dataset:** Structural Load Data
- ▶ **Source:** UCI Machine Learning Repository
- ▶ **Description:** Predict structural loads using random forests.

```
1 import pandas as pd
  from sklearn.ensemble import RandomForestRegressor
3
  data = pd.read_csv('structural_load_data.csv')
5 X = data.drop('load', axis=1)
  y = data['load']
7
  model = RandomForestRegressor()
9 model.fit(X, y)
  predictions = model.predict(X)
11
```

Simulation of Fluid Dynamics using Deep Learning

- ▶ **Dataset:** Fluid Dynamics Simulation Data
- ▶ **Source:** OpenFOAM
- ▶ **Description:** Simulate fluid dynamics using deep learning models.

```
import tensorflow as tf
2
model = tf.keras.Sequential([
4     tf.keras.layers.Conv2D(32, (3,3), activation='relu',
        input_shape=(64,64,1)),
        tf.keras.layers.Flatten(),
6     tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(1)
8 ])
10 model.compile(optimizer='adam', loss='mean_squared_error')
```


Thanks ...

- ▶ Office Hours: Saturdays, 3 to 5 pm (IST); Free-Open to all; email for appointment to yogeshkulkarni at yahoo dot com
- ▶ Call + 9 1 9 8 9 0 2 5 1 4 0 6



(<https://www.linkedin.com/in/yogeshkulkarni/>)



(<https://medium.com/@yogeshharibhaukulkarni>)



(<https://www.github.com/yogeshhk/>)